

Lab Sheet # L12

Instructions:

- Submit *.sql file, named as Lab12_6XXXXXX.sql to LAs or instructors prior to My Courses.
- If students cannot finish within the class time, students can submit until the beginning of the next class.

Self-study:

<https://www.mysqltutorial.org/mysql-triggers/>

Today's Task

0. Database Setup:

Import the database by executing the **classicmodels_db.sql** file. (Same as Lab2)

	Easy Task: Basic trigger creation
1.	Create an AFTER INSERT trigger named trg_after_insert_customer that logs every new row inserted into customers. Log table: customer_insert_log log_id INT customerNumber INT customerName VARCHAR(255) log_time DATETIME
2.	Create an AFTER INSERT trigger named trg_after_insert_payment that logs every new row inserted into payments. Log table: payment_insert_log log_id INT customerNumber INT, checkNumber VARCHAR(50) paymentDate DATE amount DECIMAL(10,2) log_time DATETIME
3.	Create an AFTER INSERT trigger named trg_after_insert_order that logs every new row inserted into orders. Log table: order_insert_log log_id INT orderNumber INT customerNumber INT orderDate DATE status VARCHAR(50) log_time DATETIME

4.	Create an AFTER DELETE trigger named trg_after_delete_order that logs every deleted row from orders. Log table: order_delete_log log_id INT orderNumber INT customerNumber INT orderDate DATE status VARCHAR(50) deleted_time DATETIME
5.	Create an AFTER UPDATE trigger named trg_after_update_product_price that logs whenever products.buyPrice is updated. Log table: product_price_update_log log_id INT productCode VARCHAR(15) productName VARCHAR(255) old_buyPrice DECIMAL(10,2) new_buyPrice DECIMAL(10,2) log_time DATETIME
6.	Create an AFTER UPDATE trigger named trg_after_update_credit_limit that logs whenever customers.creditLimit is changed. Log table: customer_credit_update_log log_id INT customerNumber INT customerName VARCHAR(255) old_creditLimit DECIMAL(10,2) new_creditLimit DECIMAL(10,2) log_time DATETIME
7.	Create an AFTER UPDATE trigger named trg_after_update_employee_office that logs whenever an employee's officeCode changes. Log table: employee_office_update_log log_id INT employeeNumber INT firstName VARCHAR(50) lastName VARCHAR(50) old_officeCode VARCHAR(10) new_officeCode VARCHAR(10) log_time DATETIME

8.	Create an AFTER UPDATE trigger named trg_after_update_order_status that logs whenever orders.status changes. Log table: order_status_update_log <pre> log_id INT orderNumber INT old_status VARCHAR(50) new_status VARCHAR(50) log_time DATETIME </pre>
9.	Create an AFTER INSERT trigger named trg_after_insert_orderdetail that logs every new row inserted into orderdetails. Log table: orderdetail_insert_log <pre> log_id INT orderNumber INT productCode VARCHAR(15) quantityOrdered INT priceEach DECIMAL(10,2) orderLineNumber SMALLINT log_time DATETIME </pre>
10.	Create an AFTER UPDATE trigger named trg_after_update_stock that logs whenever products.quantityInStock changes. Log table: product_stock_update_log <pre> log_id INT productCode VARCHAR(15) productName VARCHAR(255) old_quantityInStock INT new_quantityInStock INT log_time DATETIME </pre>
	Medium Task: Trigger with OLD / NEW and automatic actions
11.	Create a BEFORE INSERT trigger named trg_before_insert_product_price on products that automatically changes buyPrice to 1.00 if the inserted value is less than or equal to 0.
12.	Create a BEFORE INSERT trigger named trg_before_insert_payment_amount on payments that automatically changes amount to 1.00 if the inserted value is less than or equal to 0.
13.	Create a BEFORE INSERT trigger named trg_before_insert_order_status on orders that sets status to 'Pending' if the inserted status is NULL.

14.	Create an AFTER INSERT trigger named trg_after_insert_orderdetail_reduce_stock that decreases products.quantityInStock according to the inserted orderdetails.quantityOrdered .
15.	Create an AFTER DELETE trigger named trg_after_delete_orderdetail_restore_stock that increases products.quantityInStock according to the deleted orderdetails.quantityOrdered .
16.	Create an AFTER UPDATE trigger named trg_after_update_orderdetail_adjust_stock that adjusts products.quantityInStock based on the difference between OLD.quantityOrdered and NEW.quantityOrdered .
17.	Create an AFTER UPDATE trigger named trg_after_update_credit_limit_history that stores both old and new creditLimit values when a customer's credit limit changes. Log table: customer_credit_history_log log_id INT customerNumber INT customerName VARCHAR(255) old_creditLimit DECIMAL(10,2) new_creditLimit DECIMAL(10,2) changed_time DATETIME
	Hard Task: Multiple triggers and trigger-based business logic
18.	Create two AFTER INSERT triggers on payments. - The first trigger logs the inserted payment into a payment log table. (payment_insert_log) - The second trigger updates a summary table that stores total payment amount per customer. Use FOLLOWS or PRECEDES to control the execution order. Summary table: payment_summary_by_customer customerNumber INT total_amount DECIMAL(12,2) last_payment_date DATE last_checkNumber VARCHAR(50) updated_time DATETIME

19.	Create trigger(s) to maintain a table named product_sales_summary with columns such as: productCode, totalQtySold The summary must be automatically updated when rows in orderdetails are inserted, updated, or deleted. Summary table: product_sales_summary productCode VARCHAR(15) totalQtySold INT updated_time DATETIME
20.	Create a trigger-based workflow rule on orders that automatically fixes invalid shipped date data. Example rule: • if shippedDate is earlier than orderDate • automatically set shippedDate = orderDate Use a BEFORE INSERT or BEFORE UPDATE trigger.

Important submission note

Students must test every trigger before submission to confirm that it works correctly.

For each trigger, students should verify at least these points:

- The trigger can be created successfully without syntax error.
- The trigger runs on the correct event (INSERT, UPDATE, or DELETE).
- The trigger produces the expected result on the related table or log table.
- The trigger handles the intended data change correctly using OLD and/or NEW when required.

